Programmazione

Prof. Marco Bertini
marco.bertini@unifi.it
http://www.micc.unifi.it/bertini/

Classes and objects

Why abstraction ?

- Helps in modeling the problem, separating between necessary and unnecessary details
- We want to obtain a separation between:
  - operations performed on data
  - representation of data structures and algorithms implementation
- abstraction is the structuring of a nebulous problem into well-defined entities by defining their data and (coupled) operations.

ADT (Abstract Data Type)

- An ADT is a specification of a set of data and the set of operations (the ADT's interface) that can be performed on the data.
- It is abstract in the sense that it is independent of various concrete implementations.
- When realized in a computer program, the ADT is represented by an interface, which shields a corresponding implementation. Users of an ADT are concerned with the interface, but not the implementation, that can change in the future.

ADT (Abstract Data Type) - cont.

Stack ADT

Data	Operations
data=<d_1,d_2,...,d_n)	push(x)
n	pop()
Top-of-stack	top()

User

Why encapsulation ?

- The principle of hiding the used data structure and to only provide a well-defined interface is known as **encapsulation**.
- The separation of data structures and operations and the constraint to only access the data structure via a well-defined interface allows you to choose data structures appropriate for the application environment.

Why classes ?

- A class is an actual representation of an ADT: it provides implementation details for the data structure used and operations.
- Recall the important distinction between a class and an object:
  - A class is an abstract representation of a set of objects that behave identically.
  - Objects (i.e. variables) are instantiated from classes.
- classes define properties and behaviour of sets of objects.

Classes and objects

- A class is the implementation of an abstract data type (ADT). It defines attributes and methods which implement the data structure and operations of the ADT, respectively.
- An object is an instance of a class. It can be uniquely identified by its name and it defines a state which is represented by the values of its attributes at a particular time.
- The behaviour of an object is defined by the set of methods which can be applied on it.

Procedural programming

- There's a division between data and operations on data
- The focus of procedural programming is to break down a programming task into a collection of variables, data structures, and subroutines
- When programming in C we focus on data structures and functions

OO programming

- In object-oriented programming the focus is to break down a programming task into objects and interactions between objects.
- An object is associated to data and operations on its data, e.g.:
  - an object "GameCharacter" has an internal data representing position on a map and an operation that changes the position.

Why C++ classes ?

- A C++ class can provide information hiding:
  - Hides the internal representation of data
  - Hides the implementation details of operations
- The class acts like a black box, providing a service to its clients, without opening up its code so that it can be used in the wrong way

Open-Closed Principle

- Encapsulation is a key technique in following the Open-Closed principle:
  - classes should be open for extension and closed for modification
- We want to allow changes to the system, but without requiring to modifying existing code

Open-Closed Principle

- Encapsulation is a key technique in following the Open-Closed principle:
  - c
    c
- We
  but
  coc

Open-Closed Principle:

Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.

- Bertrand Meyer

Open-Closed Principle - cont.

- if a class has a particular behaviour, coded the way we want, if nobody can change the class code we have **closed it for modification**.
- but if, for some reasons, we have to extend that behaviour we can let to extend the class to override the method and provide new functionality. The class is **open for extension**.
- We'll see how inheritance and composition will help us to follow this principle.

Open-Closed Principle - cont.

"A module will be said to be **closed** if it is available for use by other modules. This assumes that the module has been given a well-defined, stable description (the interface in the sense of information hiding)."
- Bertrand Meyer

- But if, for some reasons, we have to extend that behaviour we can let to extend the class to override the method and provide new functionality. The class is **open for extension**.
- We'll see how inheritance and composition will help us to follow this principle.

Open-Closed Principle - cont.

"A module will be said to be **closed** if it is available for use by other modules. This assumes that the module has been given a well-defined, stable description (the interface in the sense of information hiding)."
- Bertrand Meyer

"A module will be said to be **open** if it is still available for extension. For example, it should be possible to add fields to the data structures it contains, or new elements to the set of functions it performs."
- Bertrand Meyer

- We'll see how inheritance and composition will help us to follow this principle.

Why OCP ?

- Here's a small example (in C) that shows a case in which the code is not closed to modifications. We'll see how, using inheritance and abstractions, we can solve

Why OCP ?

```
enum ShapeType {circle, square};

struct Shape {
  ShapeType itsType;
};

struct Circle {
  ShapeType itsType;
  double itsRadius;
  Point itsCenter;
};

struct Square {
  ShapeType itsType;
  double itsSide;
  Point itsTopLeft;
};

// These functions are implemented elsewhere
void drawSquare(struct Square*);
void drawCircle(struct Circle*);

typedef struct Shape *ShapePointer;
```

```
void drawAllShapes(ShapePointer list[], int n)
{
  int i;
  for (i=0; i<n; i++) {
    struct Shape* s = list[i];
    switch (s->itsType) {
      case square:
        drawSquare((struct Square*)s);
        break;
      case circle:
        drawCircle((struct Circle*)s);
        break;
    }
  }
}
```

Why OCP ?

```
enum ShapeType {circle, square};

struct Shape {
  ShapeType itsType;
};

struct Circle {
  ShapeType itsType;
  double itsRadius;
}

st

}

//                                  e
void drawSquare(struct Square*);
void drawCircle(struct Circle*);

typedef struct Shape *ShapePointer;
```

```
void drawAllShapes(ShapePointer list[], int n)
{
  int i;
  for (i=0; i<n; i++) {
    struct Shape* s = list[i];
    switch (s->itsType) {
      case square:
        drawSquare((struct Square*)s);
        break;
      case circle:
        drawCircle((struct Circle*)s);
        break;
    }
  }
}
```

It does not conform to the open-closed principle because it cannot be closed against new kinds of shapes.

If I wanted to extend this function to be able to draw a list of shapes that included triangles, I would have to modify the function.

In fact, I would have to modify the function for any new type of shape that I needed to draw.

Class identification

- Identify real world objects or entities as potential classes of software objects
- The usual approach is to think about the real world objects that exist in the application domain which is being programmed. Instead of thinking about what processing has to be done, as we so often do in procedural programming, we instead think about what things exist.

Class identification - cont.

- Identify groups of objects that behave similarly, that can be implemented as classes
- Classes are specifications for objects
- Delay decisions about implementation details, such as what data and operations will apply to objects, until we have a clear idea of what classes of object will be required

Class identification - cont.

- Begin class modeling by identifying candidate classes - an initial list of classes from which the actual design classes will emerge.
- A rule of the thumb to identify candidate classes: identify the noun and noun phrases, verbs (actions) and adjectives (attributes) from the use cases and problem description
  - there are more formal methods to identify (e.g. CRC cards, use cases) and represent (e.g. UML) classes

C++ Classes

- A C++ class extends the concept of C structs
- It collects together a group of variables (attributes or data members) that can be referenced to using a collective name and a symbolic identifier
- It can have functions (methods or function members) that operate within the context of the class
- It defines a data type: we can create instances (objects)

Class definition

- Use the keyword class, e.g.:

```
class IntStack {
  bool push(int value);
  bool pop(int* pValue);
  void init(int size);

  int TOS;
  int* buffer;
  int size;
}; // do NOT forget the ; !
```

Class definition

- Use the keyword class, e.g.:

```
class IntStack {
  bool push(int value);
  bool pop(int* pValue);
  void init(int size);

  int TOS;
  int* buffer;
  int size;
}; // do NOT forget the ; !
```

Note: this class is still missing something and would NOT work as expected... we'll see a more complete implementation in a few slides

A C stack implementation

```
struct IntStack {
   int TOS;
   int *buffer;
   int size;
};
bool push(struct IntStack *ptr, int value) {
...
}
bool pop(struct IntStack *ptr, int *pValue) {
...
}
```

Self reference

- An object can refer to itself using the keyword `this`
- An object implicitly uses `this` when it refers to a method or attribute

```
IntStack::push(int value) {
  // ...
  if (TOS > size)
    // ...
}
```

```
IntStack::push(int value) {
  // ...
  if (this->TOS > this->size)
    // ...
}
```

Access level

- All the members of a `struct` are visible as soon as there's a reference to the structure, while in a `class` it is possible to differentiate the access as `public`, `private` and `protected`. The default class access is `private`.
- We can design better the "interface" of the class, i.e. decide what can is hidden and what is visible in the class (encapsulation). We can decouple classes.

Access levels

- `public`: a public member is visible to anyone who has an address or a reference to the object
- `private`: a private member is visible only to the methods of the class in which it is defined
- `protected`: a protected member is visible only to the methods of the class in which it is defined, and in the derived classes (through and inheritance mechanism)

Access levels: example

```
class IntStack {
public:
      bool push(int value);
      bool pop(int* pValue);
      void init(int size);

private:
      int TOS;
      int* buffer;
      int size;
};
bool IntStack::push(int value) { /* ... */ };
```

Access levels: rules of the thumb

- Always use explicit access control
- Do not have public data members
  - use public methods to set/get their values
    - many IDEs can create these methods automatically

Getter/setter creation in CLion

```
1   #ifndef GAMECHARACTER_H
2   #define GAMECHARACTER_H
3
4
5   class GameCharacter {
6
7   private:
8       int posX;
9       int posY;
10
11  };
12
13
14  #endif //GAMECHARACTER_H
15
```

Getter/setter creation in CLion

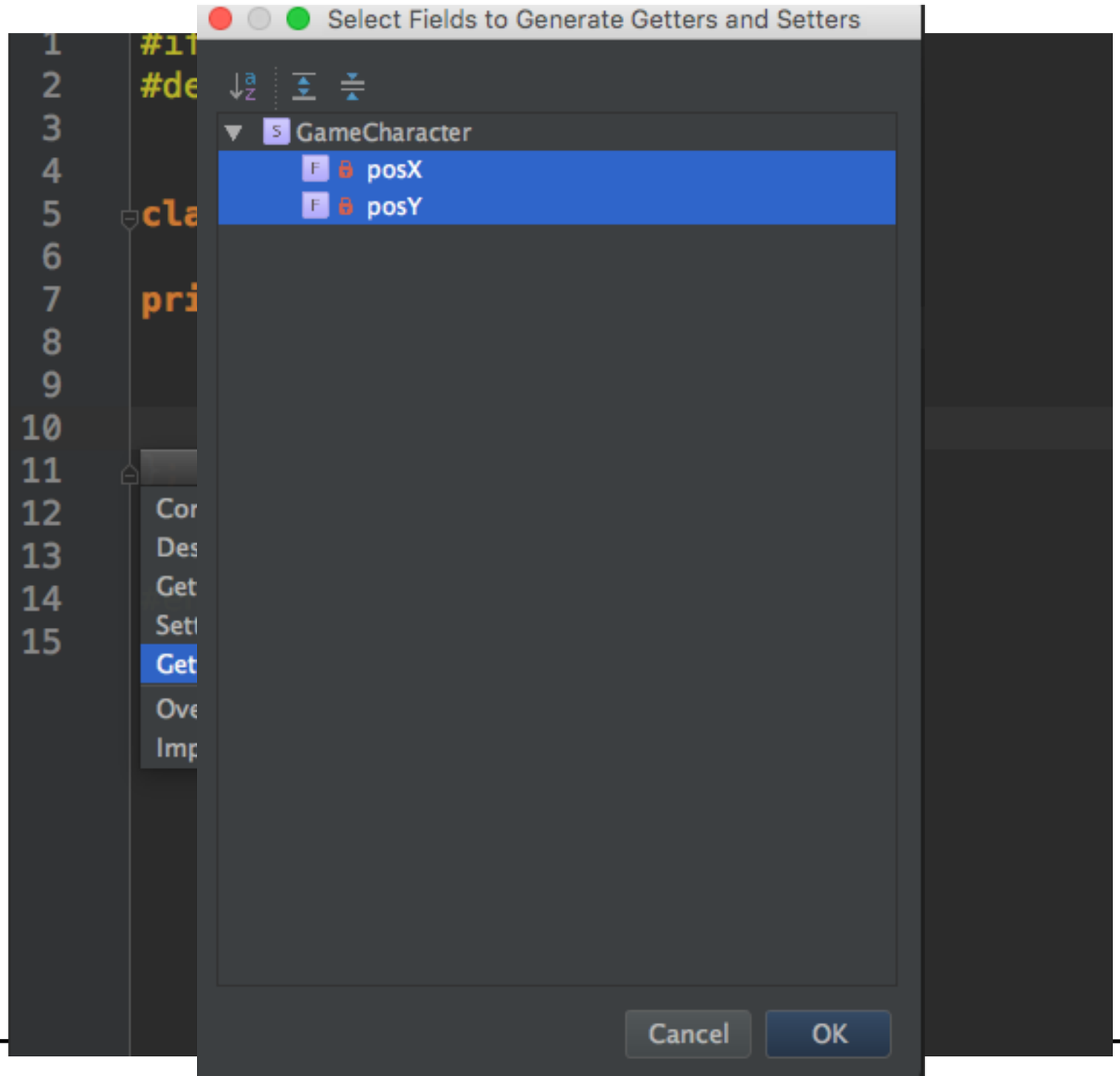
```
1   #ifndef GAMECHARACTER_H
2   #define GAMECHARACTER_H
3
4
5   class GameCharacter {
6
7   private:
8       int posX;
9       int posY;
10
11  };
12
13
14  #endif //GAMECHARACTE
15
```

- Copy Reference ⌥⇧⌘C
- Paste ⌘V
- Paste from History... ⇧⌘V
- Paste Simple ⌥⇧⌘V
- Column Selection Mode ⇧⌘8
- Find Usages ⌥F7
- Refactor ▸
- Folding ▸
- Go To ▸
- Generate... ⌘N
- Local History ▸
- Compare with Clipboard
- File Encoding
- Create Gist...

Getter/setter creation in CLion

```
1   #ifndef GAMECHARACTER_H
2   #define GAMECHARACTER_H
3
4
5   class GameCharacter {
6
7   private:
8       int posX;
9       int posY;
10
11  Generate
12  Constructor
13  Destructor
14  Getter                    CTER_H
15  Setter
    Getter and Setter
    Override Functions...   ^O
    Implement Functions...  ^I
```

Getter/setter creation in CLion

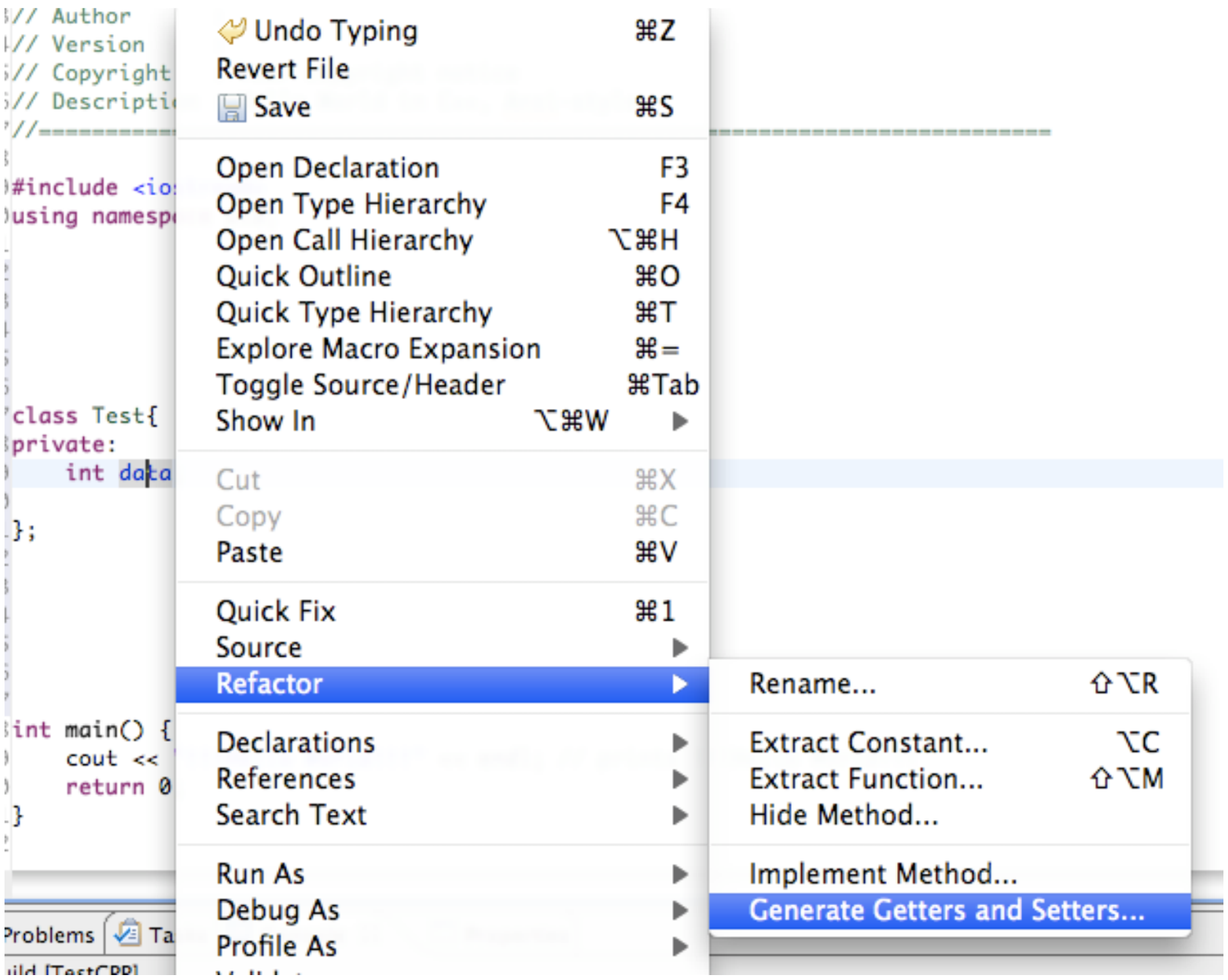


Getter/setter creation in CLion

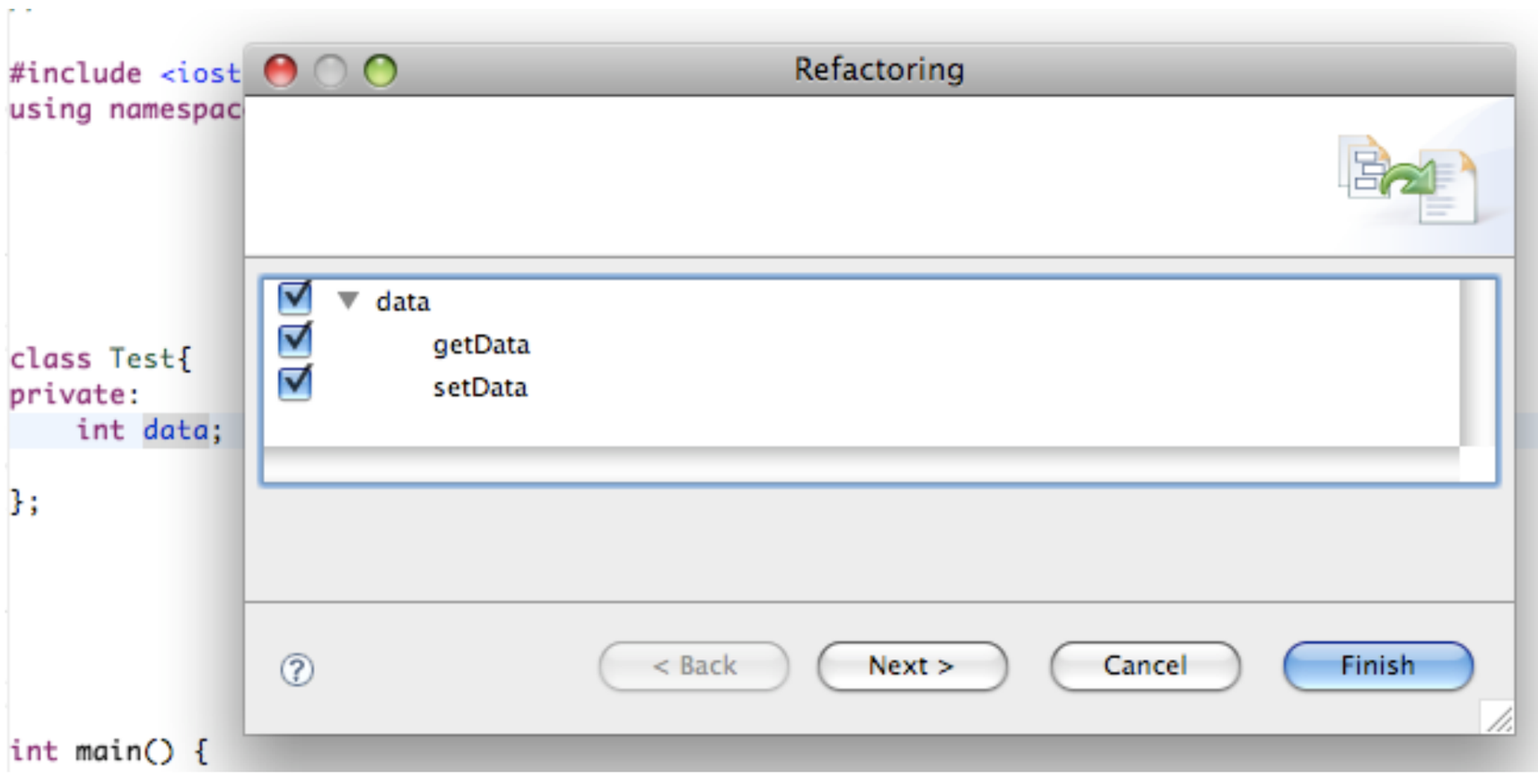
```
4
5   class GameCharacter {
6
7   private:
8       int posX;
9       int posY;
10
11  public:
12      int getPosY() const {
13          return posY;
14      }
15
16      void setPosY(int posY) {
17          GameCharacter::posY = posY;
18      }
19
20      int getPosX() const {
21          return posX;
22      }
23
24      void setPosX(int posX) {
25          GameCharacter::posX = posX;
26      }
27  };
```

Getter/setter creation in Eclipse

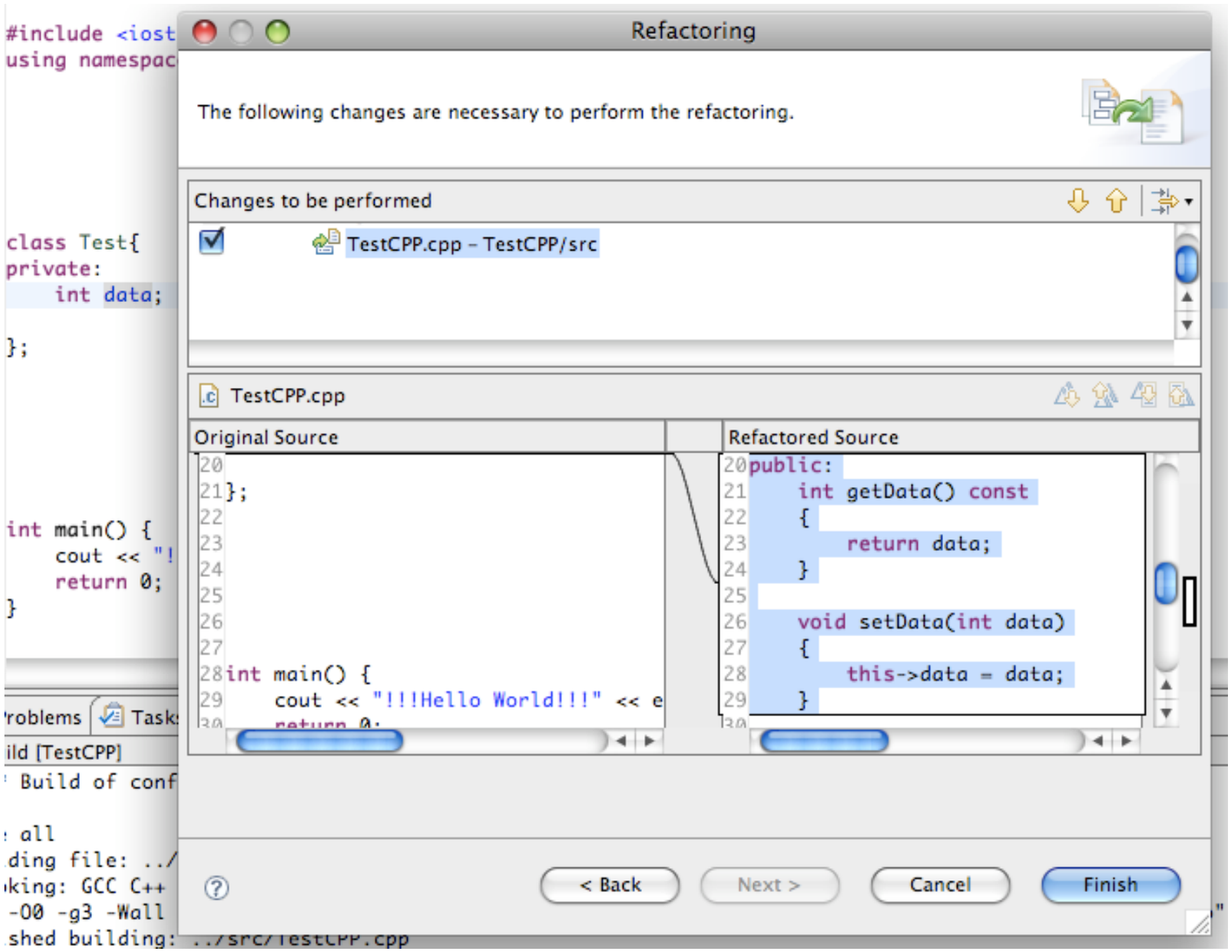
Getter/setter creation in Eclipse



Getter/setter creation in Eclipse



Getter/setter creation in Eclipse



Getter/setter creation in Eclipse

```
class Test{
private:
    int data;
public:
    int getData() const
    {
        return data;
    }

    void setData(int data)
    {
        this->data = data;
    }

};
```

Method implementation

- The methods are usually defined (implemented) in the .cpp files: add the class name in front of the method, e.g.:

```
bool IntStack::push(data value) {
// code to implement the method
}
```

- We can implement them also in the header (inline), but usually this is done only if they are very short (e.g. ~5-7 lines)

Attributes

- A method may access the attributes of the class: the attributes are visible within the methods
  - this greatly reduces the complexity of C "interfaces": compare the C++ implementation with a C implementation
- The attributes maintain the "state" of the object

Attributes - cont.

- The attributes are a sort of a context shared by the methods (that's why interfaces are simpler).
- However, the methods are more coupled with the attributes.
- It's well worth to pay for this price, if the classes have been designed to have cohesive responsibilities*

*a responsibility is something that a class knows or does.

How to use methods and attributes ?

- Class members can be referenced to analogously to struct members:

```
<var>.member_name
<expr_addr>->member_name
```

but taking into account their visibility, defined by the access level, e.g. `public`.

How to use methods and attributes: example

```
IntStack s;
IntStack* pS;

...
pS = &s;
...
s.push(3);
...
pS->push(8);
```

Overloading

- We can define more than one method with the same name and return type, but with different (number and types) parameters (signature). The method is said to be <u>overloaded</u>.
- Commonly used to provide alternate versions of functions to be used in different situations
  - the same abstract operation may have different concrete implementations

Overloading - cont.

- We can not just change the return value: the compiler should always check the type of the variable where we put the value... and what if we discard it ?

- The compiler will create a different code segment and symbol (through name mangling), obtained extending the method name with suffixes related to the types of the parameters

```
class Test{
private:
    int data;
public:
    int getData() const
    {
        return data;
    }

    void setData(int data)
    {
        this->data = data;
    }
    void setData(int data, float mul)
    {
        this->data = data*mul;
    }

};
```

Terminal — b

```
nirvana:Debug bertini$ nm TestCPP
0000200c D _NXArgc
00002008 D _NXArgv
00001ec4 s __GLOBAL__I_main
00001e6c s __Z41__static_initialization_and_
00001e2a T __ZN4Test7setDataEi
00001e3a T __ZN4Test7setDataEif
00001e1c T __ZNK4Test7getDataEv
         U __ZNSolsEPFRSoS_E
         U __ZNSt8ios_base4InitC1Ev
```

Operator overloading

- It is possible to overload also operators, not only methods (in real life: + is an operator used for integers, real numbers, complex numbers...)
- Overload operators when it really makes sense
  - e.g. overload == to compare strings, do not overload * for strings...
  - Some OO languages do not allow operator overloading, most notably Java*

*sort of... String + is operator overload

Operator overloading - cont.

- Operators are overloaded so that the objects behave as primitive types. New operators cannot be created, only the functionality of existing operators on objects can be modified
- If you overload + do NOT expect that += is given automatically !
  Define also that operator !
- Often operators are just friends... (more later)

Operator overloading - cont.

```
class Array {
public:
  Array(int size); // constructor
  bool operator ==(const Array& right) const; // the method can't
                                              // modify anything
  // ... other members
private:
  int size;
  int* data;  // pointer to first element of array
};
bool Array::operator==(const Array& right) const {
  if ( size != right.size )  // start checking the size of the arrays
    return false;
  // then check the whole content of arrays
  for ( int i=0; i < size; i++ ) {
    if ( data[i] != right.data[i] )
      return false;
  }
  return true;  // both size and content are equal
}
```

Operator overloading - cont.

Do not worry about these, you'll learn about them soon

```
class Array {
public:
  Array(int size); // constructor
  bool operator ==(const Array& right) const; // the method can't
                                              // modify anything
  // ... other members
private:
  int size;
  int* data;  // pointer to first element of array
};
bool Array::operator==(const Array& right) const {
  if ( size != right.size )  // start checking the size of the arrays
    return false;
  // then check the whole content of arrays
  for ( int i=0; i < size; i++ ) {
    if ( data[i] != right.data[i] )
      return false;
  }
  return true;  // both size and content are equal
}
```

Operator overloading - cont.

Do not worry about these, you'll learn about them soon

```
class Array {
public:
  Array(int size); // constructor
  bool operator ==(const Array& right) const; // the method can't
```

```
Array A;
Array B;
if (A == B)
    // do something
```

equivalent to

```
A.operator==(B);
```

```
    return false;
  }
  return true;  // both size and content are equal
}
```

Operator overloading - cont.

- Assignment Operator

```
T& T::operator=(T rhs) {
    // copy rhs attributes in current object attributes
    return *this;
}
```

- Binary arithmetic operators

```
class X {
  X& operator+=(const X& rhs) {
    // actual addition of rhs to *this
    return *this;
  }
};
inline X operator+(X lhs, const X& rhs) {
  lhs += rhs;
  return lhs;
}
```

Operator overloading - cont.

- Assignment Operator

```
T& T::operator=(T rhs) {
    // copy rhs attributes in current object attributes
    return *this;
}
```

- Binary arithmetic operators

```
class X {
  X& operator+=(const X& rhs) {
    // actual addition of rhs to *this
    return *this;
  }
};
inline X operator+(X lhs, const X& rhs) {
  lhs += rhs;
  return lhs;
}
```

Rule of thumb: + and its companions should be non-members, while their compound assignment counterparts (+= etc.), changing their left argument, should be a member.

- Comparison operators
  It is easy to write a different operator from an equal comparison (using a !), or a > from a < inverting arguments:

```
bool operator==(const X& lhs, const X& rhs) { /* do actual comparison */ }
bool operator!=(const X& lhs, const X& rhs) {return !operator==(lhs,rhs);}

inline bool operator< (const X& lhs, const X& rhs) {
        /* do actual comparison */
}
inline bool operator> (const X& lhs, const X& rhs) {
        return  operator< (rhs,lhs);
}
```

Argument passing

- In C the argument passing mechanism is "pass by value": the value of run-time arguments are copied in the formal parameters
  - a function uses the copy of the values to carry out its computation
  - we have to use pointers to simulate a "pass by reference"
- In C++ we can pass parameters by reference

Argument passing

- In C the argument passing mechanism is "pass by value": the value of run-time arguments are copied in the formal parameters

Note: references **are not** associated with classes: you can use references in functions, as well.

to carry out its computation

- we have to use pointers to simulate a "pass by reference"

- In C++ we can pass parameters by reference

Pass by reference

- A reference is essentially a synonym (alias) in the sense that there is no copying of the data passed as the actual argument.
- It is indicated by the ampersand (&) characters following the argument base type.
- C++ call by reference and C-style simulated call by reference using pointers are similar, but there are no explicit pointers involved: no need to dereference the argument.

Pass by reference

References can be returned as method/function values:

```
T& doSomething(T& value);
T& T::doSomethingElse(T& otherValue);
```

- It is indicated by the ampersand (&) characters following the argument base type.
- C++ call by reference and C-style simulated call by reference using pointers are similar, but there are no explicit pointers involved: no need to dereference the argument.

Pass by reference - cont.

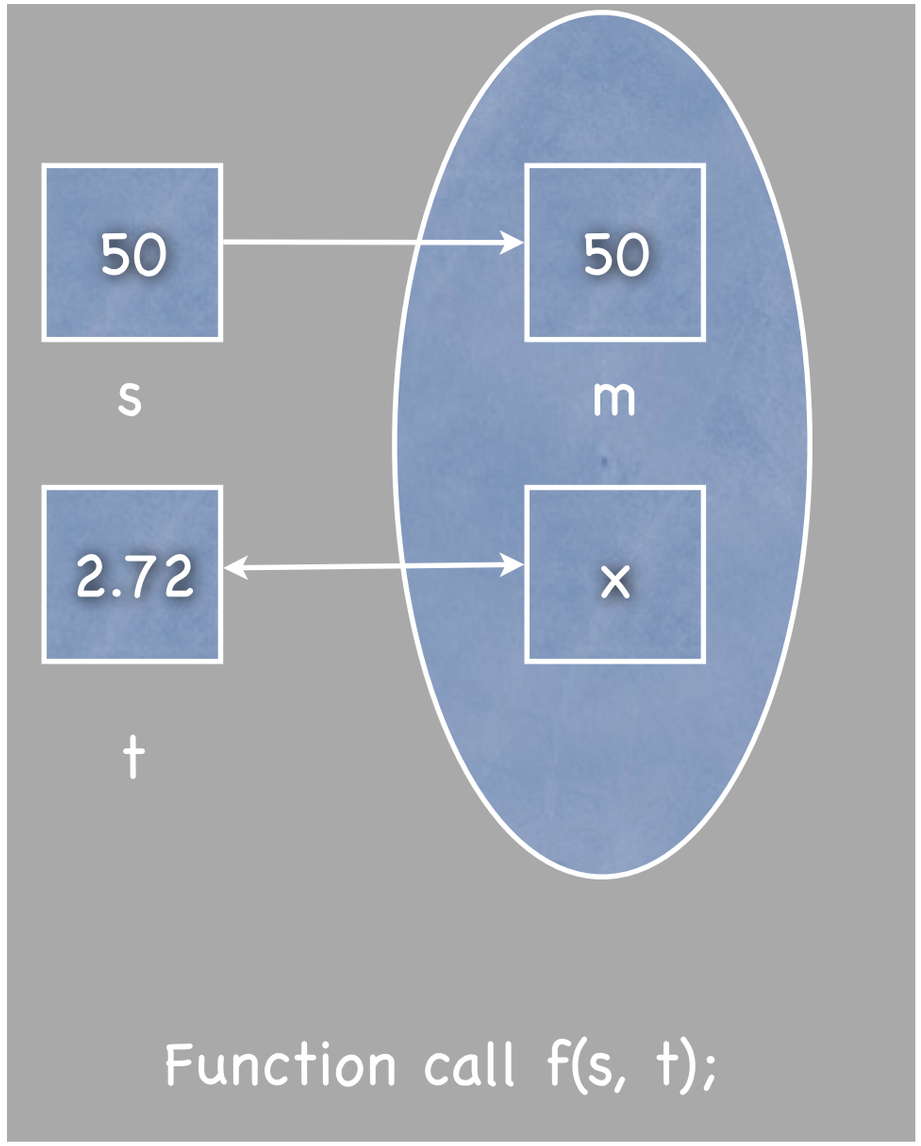
```
void add(int a, int b, int& sum) {
  sum = a + b;
}

int main() {
  int i = 1;
  int j = 2;
  int total = 0;
  cout << “total: “ << total << endl;
  add( i, j, total);
  cout << “total: “ << total << endl;
}
```

Pass by reference - cont.

```
void f(int m, double& x) {
  m = 100;
  x = 3.14;
}
int main() {
  int s = 50;
  double t = 2.72;
  f(s,t);
  return 0;
}
```

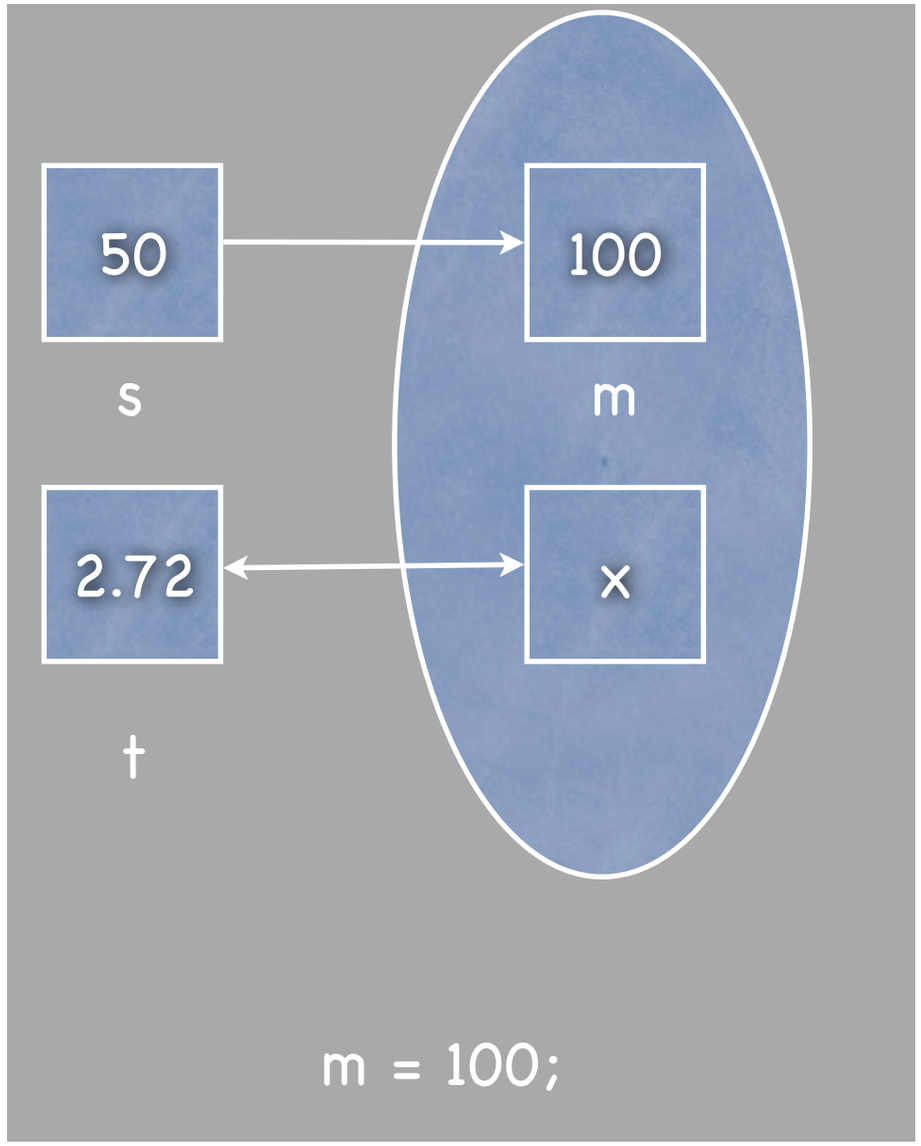
Pass by reference - cont.

```
void f(int m, double& x) {
  m = 100;
  x = 3.14;
}
int main() {
  int s = 50;
  double t = 2.72;
  f(s,t);
  return 0;
}
```



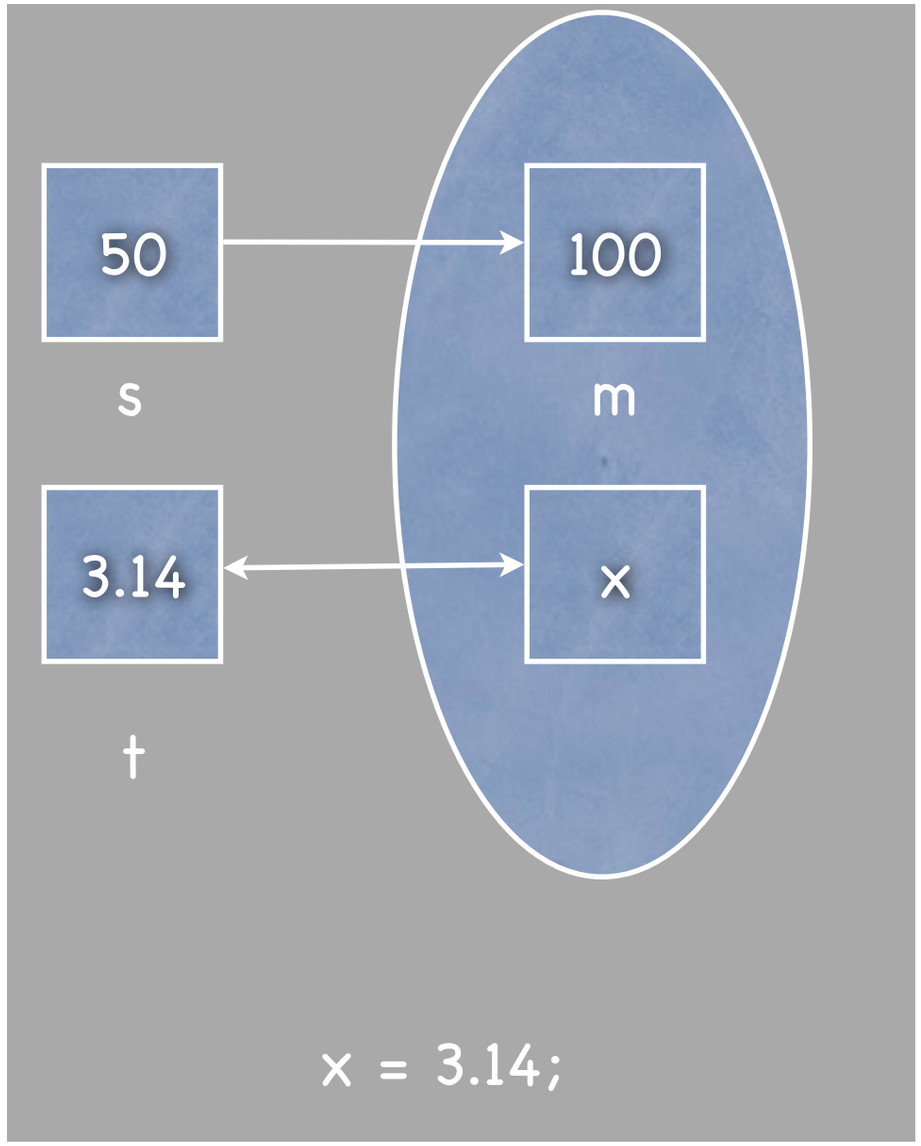
Pass by reference - cont.

```
void f(int m, double& x) {
  m = 100;
  x = 3.14;
}
int main() {
  int s = 50;
  double t = 2.72;
  f(s,t);
  return 0;
}
```



Pass by reference - cont.

```
void f(int m, double& x) {
  m = 100;
  x = 3.14;
}
int main() {
  int s = 50;
  double t = 2.72;
  f(s,t);
  return 0;
}
```



Pass by reference - cont.

- A reference can be specified as `const`: the function/method can not modify the content of the variable
  - pass large data structures that should not be modified as const references (it's fast)

```
void add(int a, int b, const int& sum) {
    sum = a + b;
}
```

```
ages Build
/Applications/CLion.app/Contents/bin/cmake/bin/cmake --build
 /Users/bertini/Library/Caches/CLion12/cmake/generated/6a4780c6/6a4780c6/Debug --target hello_world3 -
 -j 8
Scanning dependencies of target hello_world3
[ 50%] Building CXX object CMakeFiles/hello_world3.dir/main.cpp.o
[ 50%] Building CXX object CMakeFiles/hello_world3.dir/GameCharacter.cpp.o
/Users/bertini/Documents/presentazioni/lezioni/Programmazione 2015-2016/workspace/hello_world3/main
 .cpp:5:9: error: read-only variable is not assignable
    sum = a + b;
    ~~~ ^
1 error generated.
make[3]: *** [CMakeFiles/hello_world3.dir/main.cpp.o] Error 1
make[2]: *** [CMakeFiles/hello_world3.dir/all] Error 2
make[1]: *** [CMakeFiles/hello_world3.dir/rule] Error 2
make: *** [hello_world3] Error 2
```

Reference variables

- It is possible to have a reference variable, but it must always hold a valid reference and so must be initialised when it is created.

```
int x;
int& y=x; // reference
y=2; // also x is modified
int& z; // Error: doesn't compile ! Why ?
int *z; // pointer
z = &x; // & on the left is different from & on the right of =
*z = 3; // x is modified
```

Guidelines

- References are for parameter passing (and return values), sometimes they may be useful as local variables (but it's rare).
    - If there is no need to mutate an input parameter, use a const reference, unless I know that copying is cheaper (when in doubt use a reference).
    - If there is need to mutate an input parameter, use input reference (or take by value and return a copy, e.g. if copy is cheaper).
    - If there is need of a copy get the parameter by value.
    - Try to return by value rather than by out parameters (cheap, esp. since C++11 the compiler optimizes it)

Parameters guidelines

```
void f1(const string& s);  // OK: pass by reference to const; always cheap

void f2(string s);         // bad: potentially expensive

void f3(int x);            // OK: Unbeatable

void f4(const int& x);     // bad: overhead on access in f4()

void update(Record& r);  // OK: in/out param, assume that update writes to r
```

- Return a T& when copy is undesirable and “returning no object” isn’t needed
  - The return of a reference must not imply transfer of ownership !

Guidelines

	Cheap or impossible to copy (e.g., int, unique_ptr)	**Cheap to move** (e.g., vector<T>, string) or **Moderate cost to move** (e.g., array<vector>, BigPOD) or **Don't know** (e.g., unfamiliar type, template)	**Expensive to move** (e.g., BigPOD[], array<BigPOD>)
Out	X f()		
In/Out	f(X&)		
In	f(X)	f(const X&)	
In & retain "copy"			

- Note: since C++11 there are other options (e.g. move and rvalues), these will be studied in a future lecture.

Reference vs. pointer

Pointers	References
• Pointers are like other variables	• Must be initialised
• Can have a pointer to void	• Cannot have references to void
• Can be assigned arbitrary values	• Cannot be assigned
• It is possible to do arithmetic	• Cannot do arithmetic

Reference vs. pointer

- References are much less powerful than pointers
- However, they are much safer than pointers
- The programmer cannot accidentally misuse references, whereas it is easy to misuse pointers

Guidelines

- Prefer T* over T& when "no argument" is a valid option
  - A pointer (T*) can be a null and a reference (T&) cannot, there is no valid "null reference"
- Return a T* to indicate a position (only): do not transfer ownership
- Never (directly or indirectly) return a pointer or a reference to a local object:
  - To avoid the crashes and data corruption that can result from the use of such a dangling pointer.

Guidelines

```
int& f() {
    int x = 7;
    // ...
    return x;  // Bad: returns reference to object that
               // is about to be destroyed
}

int* f() {
    int fx = 9;
    return &fx;  // BAD
}
```

- Never (directly or indirectly) return a pointer or a reference to a local object:
  - To avoid the crashes and data corruption that can result from the use of such a dangling pointer.

John Carmack on pointers vs. references

- "NULL pointers are the biggest problem in C/C++, at least in our code. The dual use of a single value as both a flag and an address causes an incredible number of fatal issues. C++ references should be favored over pointers whenever possible; while a reference is "really" just a pointer, it has the implicit contract of being not-NULL. Perform NULL checks when pointers are turned into references, then you can ignore the issue thereafter. There are a lot of deeply ingrained game programming patterns that are just dangerous, but I'm not sure how to gently migrate away from all the NULL checking."

C++11 nullptr

- The newest C++ standard (C++11) provides a new special keyword: `nullptr` to set and to determine if a pointer is pointing to nothing.
- E.g.:

```
int *pa = &a;
pa = nullptr;
if (pa != nullptr) { }
```

C++11 nullptr

- The newest C++ standard (C++11) provides a new special keyword: `nullptr` to set and to determine if a pointer is pointing to nothing.
- E.g.:

```
int *pa = &a;
pa = nu
if (pa
```

- It means that you need:
  - a C++11 compiler (e.g. g++ version >= 4.2 or clang++ >= 3.x)
  - use the appropriate switches to activate C++11 compilation
    - e.g. "-std=c++11 -stdlib=libc++" for clang++
    - or use appropriate CMake instruction: `set(CMAKE_CXX_STANDARD 11)`

nullptr vs. NULL vs. 0

- NULL is simply defined as `0`
- `nullptr` is a fundamental type that's specific for pointers
  - can not be converted to `0` or any int...

C++11 nullptr - cont.

- Using nullptr makes code more clear and safe, e.g.:

```
void test(int *);
void test(int);

void foo()
{
 test(0);
 test(NULL);
}

// which function
// will be called ?
```

```
void test(int *)
void test(int);

test(nullptr); // clear now!
```

Type checking

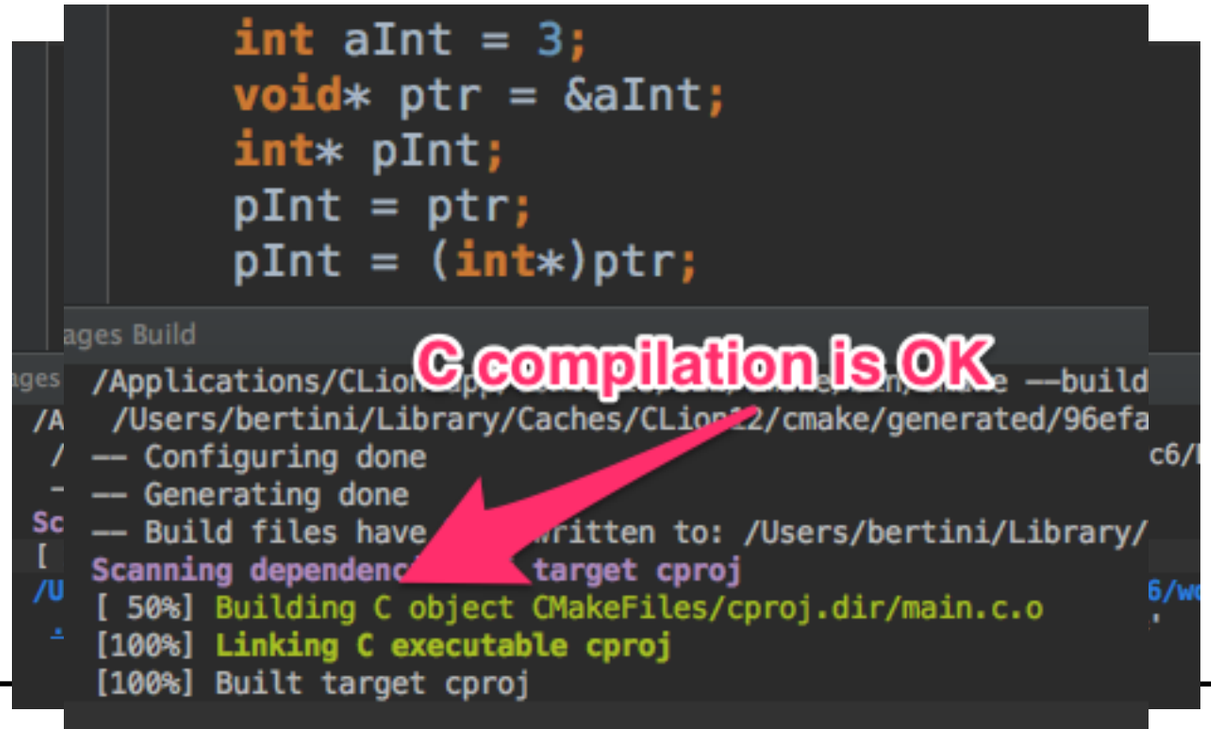
- C++ has a stricter type checking than C: depending on the parameter cast you determine the method that is executed !
- E.g.: cast void pointers when assigning them to other pointers (in C it compiles)

```
int aInt = 3;
void* ptr = &aInt;
int* pInt;
pInt = ptr;
pInt = (int*)ptr;
```

```
ges Build
/Applications/CLion.app/Contents/bin/cmake/bin/cmake --build
 /Users/bertini/Library/Caches/CLion12/cmake/generated/6a4780c6/6a4780c6/
 -j 8
Scanning dependencies of target hello_world3
[ 25%] Building CXX object CMakeFiles/hello_world3.dir/main.cpp.o
/Users/bertini/Documents/presentazioni/lezioni/Programmazione 2015-2016/w
 .cpp:5:10: error: assigning to 'int *' from incompatible type 'void *'
    pInt = ptr;
         ^ ~~~
```

Type checking

- C++ has a stricter type checking than C: depending on the parameter cast you determine the method that is executed !
- E.g.: cast void pointers when assigning them to other pointers (in C it compiles)



Object creation

- Once a class is defined we can create the instances (objects) from it, as it is done for the variables of base types.
- Creation can be static (on the stack) or dynamic (on the heap)
- The code of the methods is represented in the code segment, shared between all the instances of a class
  - each object has the address of the function that implements the methods

Dynamic object creation

- It is similar to the use of `malloc/free` functions, but syntactically simplified, using `new` and `delete`:

```
Stack* sPtr;
sPtr = new Stack;
...
delete sPtr;
```

Dynamic object creation

- The operator new automatically calculates the size of memory to be allocated, e.g.:

```
int* p = new int[5]; // gets space
                     // for 5 int
delete[] p; // if new has [] then
            // also delete must
            // have []
```

Constructors

- A member function that will be invoked when an object of that class is created. Returns no values.
  - Always has the same name as the class. Constructors generally perform some kind of initialisation on a new object. If not constructor is defined a default one is created, with no parameters .
  - Common to overload a constructor function (i.e. provide several versions) so the object can be created in a number of different ways.
- Consider how objects of a new type may be created and what constructors are needed.

Constructors - cont.

- If no constructor is defined the compiler generates a "default" constructor that takes no parameters
- The default constructor is invoked (usually) without parentheses, e.g. in the previous example:

```
sPtr = new Stack;

Stack myStack;
```

Constructors - cont.

- If no constructor is defined the compiler generates a "default" constructor that takes no parameters
- The default constructor is invoked (usually) without parentheses, e.g. in the previous example:

```
sPtr = new Stack;

Stack myStack;
```

A common mistake: `Stack myStack();`
declares a function, not an object !

Constructors - cont.

- If a class has any constructors but no default constructor, its creation will be constrained to situations handled by the constructors, e.g.

```
class B {
public:
      B(int i) { ... }
};
B b1; // illegal
B b3(123); // ok
```

Constructors - cont.

- If a class has any constructors but no default constructor, its creation will be constrained to situations handled by the constructors, e.g.

```
class B {
public:
      B(int i) { ... }
};
B b1; // illegal
B b3(123); // ok
```

If you really need this, then you'll have to write the default constructor explicitly

Constructors - cont.

- If a class has any constructors but no default constructor, its creation will be constrained to

Guideline: it is almost always right to provide a default constructor if other constructors are being defined.

```
public:
      B(int i) { ... }
};
B b1; // illegal
B b3(123); // ok
```

If you really need this, then you'll have to write the default constructor explicitly

Constructors - cont.

- There's a compact and compiler-friendly way to init attributes in a constructor:

```
class Stack {
 public:
      Stack(int s) : TOS(0), size(s), buffer(new data[s])
      {...};
 protected:
      int TOS;
      int size;
      data *buffer;
};
```

Constructors - cont.

- There's a compact and compiler-friendly way to init attributes in a constructor:

```
class Stack {
 public:
      Stack(int s) : TOS(0), size(s), buffer(new data[s])
      {...};
 protected:
      int TOS;
      int size;
      data *buffer;
};
```

Use the same order of the attributes declaration, or you'll get a compiler warning

Constructors - cont.

- Constructors are typically `public` (but not necessarily)
- If we do not want that a class is instantiated we can declare a constructor as `protected`. We can instantiate derived classes (if their constructor is `public`).
- In other cases we can declare a constructor as `private`.
  - typically its use is related to `static` methods

Constructor initializers

- Using constructor initializers is like performing definition and initialization at the same time.
  - If we do not explicitly initialize a member in the constructor initializer list, that member is default initialized before the constructor body starts executing.
  - Constant attributes and references need to be initialized explicitly.
- Guideline: use initializer lists.

Default initializers

- Since C++11 it is possible to initialize attributes when declaring them in the class definition (something that has always been possible in Java):

```
class X {
public:
  int i = 4;
  int j {5}; // alternative syntax
};
```

- Those initializers then are implicitly used in any constructor unless you specifically initialize the members in the member initializer list of that constructor, e.g.:

```
class X {
public:
  X() {} // i==4, j==5
  X(int ni) : i(ni) {} // j==5
  int i = 4;
  int j {5}; // alternative syntax
};
```

Default initializers make constructors less verbose reducing their initializer lists

- Since C++11 it is possible to initialize attributes when declaring them in the class definition (something that has always been possible in Java):

```
class X {
public:
  int i = 4;
  int j {5}; // alternative syntax
};
```

- Those initializers then are implicitly used in any constructor unless you specifically initialize the members in the member initializer list of that constructor, e.g.:

```
class X {
public:
  X() {} // i==4, j==5
  X(int ni) : i(ni) {} // j==5
  int i = 4;
  int j {5}; // alternative syntax
};
```

Explicit constructors

- In C++, the compiler is allowed to make one implicit conversion to resolve the parameters to a function.
- Therefore, if needed, C++ constructors that have just one parameter automatically perform implicit type conversion, e.g.:
  if you pass an `int` when the constructor expects a `string` pointer parameter, the compiler will add the code it must have to convert the int to a string pointer.
- You can add `explicit` to the constructor declaration to prevent these implicit conversions.

Explicit constructors - cont.

- Declaring a constructor that has multiple arguments to be explicit has no effect, because such constructors cannot take part in implicit conversions.
- However, explicit will have an effect if a constructor has multiple arguments and all, except one of the arguments, have a default value.

Explicit constructors: example

```
class A {
public:
 A();
};

class B {
public:
 explicit B(int x=0, bool b=true);
};

class C {
public:
 explicit C(B ab=B());
};


void doSomething(B objB);
void doSomethingC(C objC);
```

```
B objB1; // x and b args set to
         // default
doSomething( objB1 ); // OK

B objB2( 28 ); // OK, b arg is set
               // to default

doSomething( objB2 ); // OK
doSomething(28); // BAD: we need a
// B obj, and we do not allow
// implicit conversion
doSomething(B(28)); // OK

C objC1;
doSomethingC(objC1);
doSomethingC(objB2); // BAD, we
// need a C object, and we do not
// allow implicit conversion

doSomething("foo"); // BAD, thanks
the compiler for not allowing it
```

Explicit constructors: example

```
class A {
public:
 A();
};

class B {
public:
 explicit B(int x=0, bool b=true);
};

class C {
public:
 explicit C(B ab=B());
};
```

```
void doSomething(B objB);
void doSomethingC(C objC);
```

```
B objB1; // x and b args set to
         // default
doSomething( objB1 ); // OK

B objB2( 28 ); // OK, b arg is set
               // to default

doSomething( objB2 ); // OK
doSomething(28); // BAD: we need a
// B obj, and we do not allow
// implicit conversion
doSomething(B(28)); // OK

C objC1;
doSomethingC(objC1);
doSomethingC(objB2); // BAD, we
// need a C object, and we do not
// allow implicit conversion
```

```
doSomething("foo"); // BAD, thanks
the compiler for not allowing it
```

We need the `explicit` for these functions, not for this constructor

Explicit constructors - cont.

- It's preferable to use explicit constructor (there is even a Google C++ guideline for it)
- When designing a type (i.e. class) think about what conversions should be allowed: should you write a type conversion function or a non explicit constructor (with a single argument) ?

Delegate constructors

- A delegating constructor uses another constructor from its own class to perform its initialization.
  - Introduced in C++11
- It's useful if there's lot of duplicated code in the constructor: move it to a common constructor

Delegate constructors

- A delegating constructor uses another constructor from its own class to perform its initialization.
  - Introduced in C++11
- It's useful if there's lot of duplicated code in the constructor: move it to a common constructor

- It means that you need:
  - a C++11 compiler (e.g. g++ version >= 4.2 or clang++ >= 3.x)
  - use the appropriate switches to activate C++11 compilation
    - e.g. "-std=c++11 -stdlib=libc++" for clang++

Delegate constructors - example

```
class Sales_data {
  public:

// non delegating constructor, uses init. list
  Sales_data(std::string s, unsigned cnt, double price):
    bookNo(s), units_sold(cnt), revenue(cnt*price) { }

// remaining constructors all delegate
// to another constructor

  Sales_data(): Sales_data("", 0, 0) {}
  Sales_data(std::string s): Sales_data(s, 0,0) {}
  Sales_data(std::istream &is): Sales_data() {}

// other members
// ...
};
```

Delegate constructors - example

```
class Sales_data {
  public:

// non delegating constructor, uses init. list
  Sales_data(std::string s, unsigned cnt, double price):
    bookNo(s), units_sold(cnt), revenue(cnt*price) { }

// remaining constructors all delegate
// to another construc

  Sales_data(): Sales_
  Sales_data(std::stri
  Sales_data(std::istr

// other members
// ...
};
```

- It's much safer than writing the common code in a private method
- Can use common initialization list for const and references
- Java language has had it for years... uses `this(xxx)` instead of constructor name

Destructors

- It's a method with the name of the class preceded by ~, e.g.: ~Stack();
- It takes no parameters and has no return values, therefore it can not be overloaded.
- Called **automatically** when an object is destroyed
  - should perform housekeeping

C'tor and D'tor

```
class IntStack {
public:
  IntStack(int s);
  ~IntStack();
  //..
protected:
  int TOS;
  int size;
  int* buffer;
};
```

```
// C'tor allocates memory
IntStack::IntStack(int s) {
  TOS=0;
  size=s;
  buffer = new int[size];
}

// D'tor has to
// release memory
IntStack::~IntStack() {
  delete[] buffer;
}
```

Inversion of responsibility

- The use of `this` is essential when an object has to pass a reference of itself to another object
- A typical application is the callback: obj A gives a reference to itself that will be used by obj B to invoke a method on obj A
  - This is used to implement inversion of responsibility schemas:
    obj A does not call obj B to perform an operation but lets obj B call obj A

Inversion of responsibility - example

```
class Observer;
class Subject;
class Observer {
public:
  void update(subject*
pSubj);
  int getState() {
    return state;
  }
private:
  int state;
};

class Subject {
public:
  Subject(Observer* pObs);
  void setState(int aState);
  int getState() {
    return state;
  }
private:
  int state;
  Observer* pObs;
}
```

```
Observer::update(subject* pSubj) {
  if ( ... )  // possible condition that
              // starts an update
    this->state = pSubj->getState();
}
Subject::Subject(Observer* pObs) {
  this->pObs = pObs;
}
Subject::setState(int aState) {
  this->state  = aState;
  this->pObs->update( this );
}


int main() {
  Subject* pSubj;
  Observer* pObs;
  pObs = new Observer;
  pSubj = new Subject( pObs );
  // ...
  pSubj->setState( 10 );
  cout << “subj state: “ << pSubj->getState << endl;
  cout << “obs state: “ << pObs->getState() << endl;
}
```

Static members

- A `static` member is associated with the class, not with object (instance of the class), i.e. there's only one copy of the member for all the instances
  - extends the static variables of C
- Static data member: one copy of the variable
- Static function member: can be invoked without requiring an object

Static data members

```
class Point {
public:
  Point() {
    x=y=0;
    n++;
  }
  ~Point() {
    n--;
  }
  int count() const {
    return n;
  }
  // ...
private:
 int x,y;
 static int n; // declaration
};
```

```
// definition: must be in
// namespace scope
int Point::n = 0;

int main() {
  Point a,b;
  cout << “n: “ << a.count()
<< endl;

  cout << “n: “ << b.count()
<< endl;
}
```

Static data members

.h

```
class Point {
public:
  Point() {
    x=y=0;
    n++;
  }
  ~Point() {
    n--;
  }
  int count() const {
    return n;
  }
  // ...
private:
 int x,y;
 static int n; // declaration
};
```

```
// definition: must be in
// namespace scope
int Point::n = 0;

int main() {
  Point a,b;
  cout << “n: “ << a.count()
<< endl;

  cout << “n: “ << b.count()
<< endl;
}
```

Static data members

.h

```
class Point {
public:
  Point() {
    x=y=0;
    n++;
  }
  ~Point() {
    n--;
  }
  int count() const {
    return n;
  }
  // ...
private:
 int x,y;
 static int n; // declaration
};
```

```
// definition: must be in
// namespace scope
int Point::n = 0;

int main() {
  Point a,b;
  cout << “n: “ << a.count()
<< endl;

  cout << “n: “ << b.count()
<< endl;
}
```

.cpp

Static method members

```
class Point {
public:
  Point() {
    x=y=0;
    n++;
  }
  ~Point() {
    n--;
  }
  static int n; // declaration
  static float distance(
     const Point& a,
      const Point& b) {
    //...calc distance
  }
  // ...
private:
 int x,y;
};
```

Point.h

```
// definition: must be in
// namespace scope
int Point::n = 0;

int main() {
  // access static members even before
  // the creation of any instance of
  // the class
  cout << “n: “  << Point::n << endl;
  Point a,b;
  // set a and b coordinates
  Point::distance(a,b);
}
```

Point.cpp

Static method members

Remind to:

1. declare the static attribute
2. define the static attribute

```
class Point {
public:
  Point() {
    x=y=0;
    n++;
  }
  ~Point() {
    n--;
  }
  static int n; // declaration
  static float distance(
     const Point& a,
      const Point& b) {
    //...calc distance
  }
  // ...
private:
 int x,y;
};
```

Point.h

```
// definition: must be in
// namespace scope
int Point::n = 0;

int main() {
  // access static members even before
  // the creation of any instance of
  // the class
  cout << “n: “  << Point::n << endl;
  Point a,b;
  // set a and b coordinates
  Point::distance(a,b);
}
```

Point.cpp

Initializing static data members

- Since C++17 we can initialise static data members thanks to inline variables:

No need for definition in a `.cpp` file.

```
struct OtherType {
    static const int value = 10;
    static inline std::string className = "Hello Class";
}
```

- Note: we could initialise constant integer static fields (e.g. `value` in this example) "in place" even in C++98 !
  - The `Point` class example was just to show the concept.

Friend

- A class can allow access to its members (even if private) declaring that top-level functions (or even classes) are its friends
- Friends should only used in very special situations, e.g. I/O operator overloads where it is not desirable to provide accessor member functions.
  - It hinders encapsulation: use only when really needed.

Friend - cont.

```
class Point{
private:
  int x,y;
public:
  friend bool operator==(Point a,
                         Point b);
  Point() : x(0), y(0) {};
  //...
};

bool operator==(Point a, Point b) {
  if ( ( a.x != b.x ) ||
       ( a.y != b.y ) )
    return false;
  else
    return true;
}
```

```
int main() {
  Point p, q;
  //...
  if (p == q)
    std::cout << "p and q are equal"
              << endl;
   return 0;
}
```

Friend - cont.

- Spend some time to evaluate if you can avoid a friend... the previous code could be rewritten as:

```
class Point{
private:
  int x,y;
public:
  bool operator==(Point right);
  Point() : x(0), y(0) {};
  //...
};

bool Point::operator==(Point right) {
  if ( ( this->x != right.x ) ||
       ( this->y != right.y ) )
    return false;
  else
    return true;
}
```

```
int main() {
  Point p, q;
  //...
  if (p == q)
    std::cout << "p and q are equal"
<< endl;
   return 0;
}
```

Friend - cont.

- Spend some time t
  friend... the previou s:

Now the friend function has been transformed in a class method... it needs only 1 parameter

```
class Point{
private:
  int x,y;
public:
  bool operator==(Point right);
  Point() : x(0), y(0) {};
  //...
};

bool Point::operator==(Point right) {
  if ( ( this->x != right.x ) ||
       ( this->y != right.y ) )
    return false;
  else
    return true;
}
```

```
int main() {
  Point p, q;
  //...
  if (p == q)
    std::cout << "p and q are equal"
<< endl;
   return 0;
}
```

Inner class

- A inner class or nested class is a class declared entirely within the body of another class or interface. An instance of an inner class cannot be instantiated without being bound to a top-level class.
- Inner classes allow for the object orientation of certain parts of the program that would otherwise not be encapsulated into a class.

Inner class - cont.

- C++ nested classes are in the scope of their enclosing classes.
- Before C++11 nested class can use only type names, static members, and enumerators from the enclosing class, without qualifying the name
  (other classes that are not one of its enclosing classes have to qualify its name with its enclosing class's name).
- Since C++11 nested classes are treated as friends of enclosing classes, and by using explicit pointers, references, and object names, can access all the members.

Inner class - cont.

```
int x,y; // globals
class enclose { // enclosing class
private:
    int x;            // NOTE: private members
    static int s;
 public:
    struct inner {   // nested class
        void f(int i) {
            x = i;   // Error: can't write to non-static
                     // enclose::x without instance
            s = i;   // OK: can assign to the static enclose::s
            ::x = i; // OK: can assign to global x
            y = i;   // OK: can assign to global y
        }
        void g(enclose* p, int i) {
            p->x = i; // OK: assign to enclose::x
        }
    };
};
```

Inner class

- Let an outer class access the members of the nested class using `friend`

```
class Algorithm{
public:
    class AlgorithmResults{
    friend class Algorithm;
    public:
        void readAlgorithmResult();
    private:
        void writeAlgorithmResult();
    };

    void calculate(AlgorithmResults& results, Arguments...){
       //calculate stuff
       results.writeAlgorithmResult(results);
    }

};
```

Inner class

Inner classes are useful to hide implementation details:

```
class List {
public:
    List(): head(nullptr), tail(nullptr) {}
private:
    class Node {  // not exposed to users of List class
    public:
        int   data;
              Node* next;
              Node* prev;
    };
    Node*     head;
    Node*     tail;
};
```

Single Responsibility Principle

- Every object in the system should have a single responsibility, and all the object's services should be focused on carrying out that single responsibility
- A class should have only one reason to change
- A *responsibility* can be defined as a reason to change
- It's a concept related to cohesion

SRP & OCP

- Ideally, following the Open Closed Principle, means to to write a class or a method and then turn my back on it, comfortable that it does its job and I won't have to go back and change it.
- It's a a "laudable goal", but elusive in practice: you'll never reach true Open-Closed nirvana, but you can get close by following the related Single Responsibility Principle: a class should have one, and only one, reason to change.

SRP - cont

- As an example, consider a module that *compiles* and *prints* a report: the content of the report can *change*, the format of the report can *change*.
- The single responsibility principle says that these two aspects of the problem are really two separate responsibilities, and should therefore be in separate classes or modules.
- Do not **couple** two things that change for different reasons at different times.

SRP: example

- Here's a simple test to check if a class follows SRP: for each method of the class write a line that says

  The *class name* *write method here* itself.

- Adjust grammar and syntax the read aloud each line. Does it make sense ?

- If it doesn't probably the method belongs to a different class. Use common sense !

SRP: example - cont.

- Apply the method to the Automobile class:

Automobile
start() stop() changeTires(Tire[]) drive() wash() checkOil() getOil() : int

- We are still a bit far from having cars driving themselves (we may need a Driver)
- Surely they won't change their tires or wash themselves (Mechanic and CarWash may help...)
- Think very well about the meaning of the methods: getOil may simply mean that the car has a sensor

Reading material

- M. Bertini, "Programmazione Object-Oriented in C++", cap. 2, cap. 4 pp. 95-100
- B. Stroustrup, "C++, guida essenziale per programmatori" - pp.15-18, pp. 33-39
- B. Stroustrup, "C++, Linguaggio, libreria standard, principi di programmazione", cap. 3, 16, 18
- L.J. Aguilar, "Fondamenti di programmazione in C++. Algoritmi, strutture dati e oggetti" - cap. 8, 9, 12
- D.S. Malik, "Programmazione in C++" - cap. 6, 8

Credits

- These slides are (heavily) based on the material of:
  - Dr. Ian Richards, CSC2402, Univ. of Southern Queensland
  - Prof. Paolo Frasconi, IIN 167, Univ. di Firenze
  - "Head first: Object Oriented Analysis and Design", O'Reilly